

Embeddings & vector search.

Find useful neighbors.

Turn representations into retrieval.

Understand similarity, normalization, and approximate search.

🕒 18-22 min core reading · 20-30 min practice · 12 short sections

A viewer searches for “space rescue.” A useful movie description says “astronauts save a stranded pilot.” There are no shared keywords. Embeddings offer another way to compare the two: represent them as vectors, then look for useful neighbors.

WHAT YOU WILL UNDERSTAND

The main question: how can numbers represent useful relationships between texts, and what can go wrong when we search those numbers?

The practical path: understand the encoder, compare vectors, protect compatibility, build an index, and evaluate both search accuracy and relevance.

Reading path: twelve short sections, three diagrams, a vector-search explorer, and a Python workbook. Deeper notes are optional.

1. A vector is a representation, not a verdict

An **embedding** maps an item into a numerical vector. For text retrieval, an **encoder** turns a query or passage into a fixed number of coordinates. Similarity in that space is intended to reflect relationships useful for its training task.

An embedding is not the text's ID, a set of word counts, or a factuality score. Unlike the sparse TF-IDF coordinates in Chapter 16, a dense learned vector usually has many nonzero entries whose meaning is distributed across coordinates. "Coordinate 42 means adventure" is generally not a valid interpretation.

A good retrieval encoder may place "space rescue" near "astronauts save a stranded pilot." That is learned behavior to test, not a guarantee of embeddings in general. Domain, language, wording, negation, and the model's training objective matter.

For this chapter's explorer, we **choose three-dimensional vectors by hand** for the eight familiar movies and three preset queries. They make geometry inspectable; they are not outputs of a neural encoder. A successful result on these authored coordinates cannot establish that a trained model outperforms keyword search.

2. Understand what the encoder learns

Many retrieval encoders learn from related and unrelated text pairs. A training example might pair "space rescue" with a useful summary as a **positive**, and with an unrelated café summary as a **negative**. A contrastive objective can reward higher similarity for the positive than for alternatives.

The learned parameters determine the representation. At inference time, the fixed encoder calculates a new vector; saving that vector to an index does not train the encoder again. Methods differ in their losses, training data, pooling, and use of negatives. [Sentence-BERT](#) is a primary example of learning reusable sentence representations for similarity comparisons.

A **bi-encoder** represents the query and document separately. This lets us precompute document vectors and compare them with many later queries. A query and its answer need not say the same thing: a retrieval model must capture usefulness for the intended task, not merely paraphrase similarity.

▼ Go deeper: tokens, pooling, and hard negatives

A text encoder commonly tokenizes input, computes contextual token representations, then uses a pooling or selection rule to produce one passage vector. A token embedding, a whole-passage embedding, and an LLM's generated answer are different objects. Arbitrarily averaging hidden states from any language model does not ensure good retrieval.

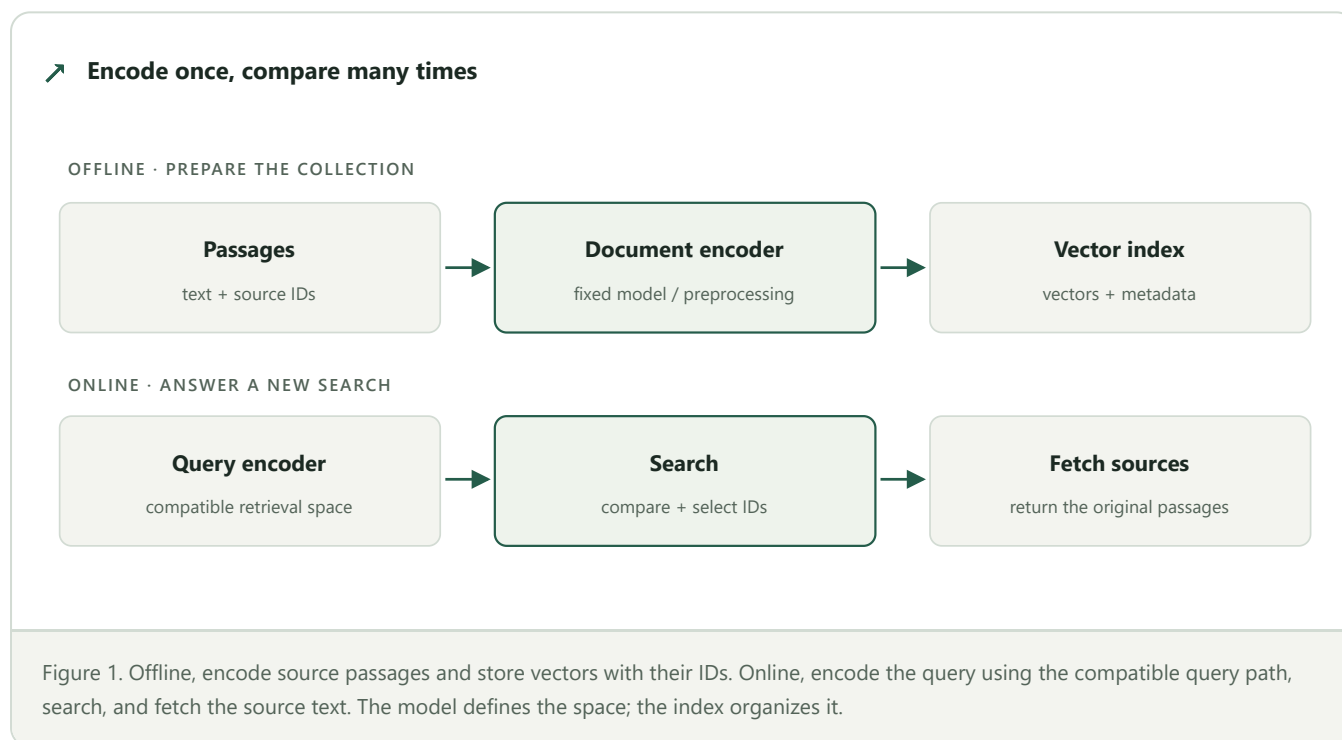
A **hard negative** looks related but should not answer the query, such as a rescue mission on a city road for an outer-space request. It can teach finer distinctions. But a supposedly negative example may actually be relevant: false negatives introduce conflicting supervision. Training data quality and the intended retrieval task matter as much as the loss's name.

Input limits also matter. If a long synopsis is truncated, its rescue scene may never reach the encoder. Pooling compresses information too; a single vector does not retain every detail. Chapter 18 introduces passage boundaries and chunking in a full RAG pipeline.

3. Keep queries and documents in a compatible space

Both sides must follow a compatible representation contract: model and version, input preparation, dimensionality, query/document roles, and any normalization. Some retrieval models require different prompts or paths for queries and passages. Follow that model's documented recipe rather than assuming the inputs are interchangeable. The [Sentence Transformers semantic-search guide](#) distinguishes symmetric similarity from short-query-to-passage retrieval.

Equal dimensions are necessary, but not sufficient. Two unrelated encoders can each return 384 numbers while organizing those coordinates differently. Comparing a query from one with stored vectors from the other has no established semantic meaning.



Store each vector with a stable source ID, source text or pointer, content revision, and embedding version. Re-encode changed text. When changing the encoder or its representation recipe, build a compatible replacement index and validate it before switching queries to it. The vector index organizes representations; it does not repair incompatible ones.

Our workbook records an explicit `space_id` and rejects mismatched IDs, wrong dimensions, nonfinite coordinates, and zero-length vectors. This catches declared contract failures, but a matching string cannot prove that someone actually used the claimed encoder.

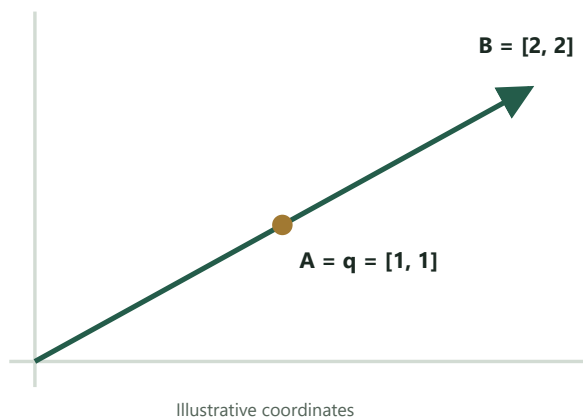
4. Choose what close means

Chapter 6 introduced the arithmetic. Now it becomes a retrieval policy. For query vector q and document vector x :

Comparison	Formula	Preferred result
Dot product	$q \cdot x = \sum_i q_i x_i$	Larger
Cosine similarity	$(q \cdot x) / (\ q\ \ x\)$	Larger
Euclidean distance	$\sqrt{\sum_i (q_i - x_i)^2}$	Smaller

The **norm**, $\|x\| = \sqrt{\sum_i x_i^2}$, is a vector's length. Cosine compares direction after dividing out lengths. Dot product is influenced by both direction and magnitude. Euclidean distance measures separation in the coordinate space.

➤ Direction and length are different information



Same direction; different magnitude

$$\cos(q, A) = \cos(q, B) = 1$$

$$q \cdot A = 2; q \cdot B = 4$$

$$\text{distance}(q, A) = 0; \text{distance}(q, B) = \sqrt{2}$$

Unit normalization makes A and B identical.

Figure 2. These two-dimensional teaching arrows are not learned features. A and B point in the same direction, so both have cosine 1 with q , but B has twice the dot product.

For our space query $q = [1, 0, 0]$, Quiet Orbit has $x = [0.9, 0.1, 0]$. Its dot product is 0.9, its norm is $\sqrt{0.82}$, and its cosine is $0.9 / \sqrt{0.82} \approx \mathbf{0.9939}$. Its Euclidean distance is $\sqrt{0.02} \approx \mathbf{0.1414}$.

City Sprint's authored vector is $[3, 2, 0.2]$. Its raw dot product is **3**, so it beats Quiet Orbit under that rule despite being irrelevant to the outer-space intent. That exposes a representation/metric mismatch in this fixture. Dot product is not inherently wrong: use the comparison the real model was designed and evaluated for. See the [Faiss metric reference](#).

5. Normalize when the representation calls for it

Unit normalization divides a nonzero vector by its norm: $\hat{x} = x/\|x\|$. It preserves direction and makes length equal to one. It does not turn coordinates into probabilities or improve their semantic meaning automatically.

For unit query and document vectors:

$$\hat{q} \cdot \hat{x} = \cos(q, x), \quad \|\hat{q} - \hat{x}\|^2 = 2 - 2\cos(q, x).$$

Therefore descending dot product, descending cosine, and ascending Euclidean distance give the same ordering, apart from numerical ties. Without normalization they can disagree. Our raw Euclidean order is Quiet Orbit, Unlisted Voyage, Parallel World; raw cosine swaps the last two.

Cosine ranges from -1 to 1 , but a value such as **0.99 is not a 99% chance of relevance**. Scores and useful thresholds depend on the encoder, data, and task. Even unrelated queries have nearest neighbors, so top-K alone does not prove that a good match exists.

▼ Go deeper: the identity, zero vectors, and numerical precision

Expand squared distance: $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a \cdot b$. For unit vectors, the first two terms are both 1 , giving $2 - 2a \cdot b$. This explains the ranking equivalence without needing a special library.

A zero vector has no direction, so cosine and unit normalization divide by zero. This workbook rejects norms below 10^{-12} instead of assigning a made-up score. Floating-point results can differ in their last digits; we clamp cosine to $[-1, 1]$ to handle rounding and break exact score ties by movie ID.

Some search libraries report **squared** Euclidean distance. It gives the same ordering as distance, but different values and thresholds. Our explorer displays the square root. Quantization stores approximate coordinates with fewer bits; it can change neighbors and needs measurement against an uncompressed reference.

6. Distinguish a vector index from a vector database

An **exact search** compares the query with every eligible vector, then keeps the best K . With N vectors and d coordinates, the scoring work is proportional to Nd , before result selection. For eight movies, this is straightforward and provides a useful reference answer.

A **vector index** is a data structure or algorithm for finding neighbors. A **vector database** or a database with vector support adds storage, updates, metadata queries, durability, and other operational features. Storing numbers does not itself require a separate database product, and a database cannot make poor embeddings meaningful.

Choose infrastructure based on measured scale and requirements. For example, one million 768-dimensional vectors stored as float32 values require $1,000,000 \times 768 \times 4 = \mathbf{3,072,000,000 \text{ bytes}}$, about 3.07 GB in decimal units, for coordinates alone. IDs, indexes, metadata, source text, and working memory add more. Higher dimensions increase storage and scoring work without guaranteeing better retrieval.

▼ Go deeper: filters, updates, and stale representations

A runtime requirement still belongs in metadata. Unlisted Voyage has unknown runtime, so it cannot satisfy “known runtime below 120 minutes,” however close its vector is. Keep permissions and other hard eligibility rules separate from similarity.

Applying filters after an approximate top-K search can leave too few results or miss eligible neighbors outside the initial candidate set. Filter-aware search, larger candidate pools, or iterative scans may help; behavior depends on the engine and index. The [pgvector documentation](#) discusses exact search, approximate indexes, and filtering tradeoffs.

Keep vector and source revisions aligned. If a synopsis changes while its old vector remains, search and displayed evidence disagree. Deletion, deduplication, model upgrades, and query caching also need explicit handling. Our workbook rebuilds a tiny index for each run; it is not a persistent storage service.

7. Trade search work for approximation deliberately

Approximate nearest-neighbor search (ANN) tries to find close vectors while examining fewer candidates. It may miss a vector that the exact search would return. This approximation error is separate from the embedding model’s semantic errors.

One approach partitions vectors into cells around centroids and searches only a few promising cells. **IVF** indexes use this broad idea. Another approach, **HNSW**, searches through a graph of neighboring vectors at multiple levels. Their construction costs, memory use, and search settings differ. See the [Faiss index overview](#).

➤ A fast candidate step can miss a close neighbor

Query → compare 3 centroids → visit the selected cells

A · nearest centroid

M004, M008

2 searched candidates

B · next centroid

M001, M002, M003, M005, M007

includes a missed exact neighbor

C · remaining centroid

M006

not needed for this query’s top 3

1 cell: neighbor recall@3 = 2/3. Two cells: 3/3.

Neighbor recall checks search fidelity, not whether the movies are relevant.

Figure 3. For the authored space-rescue query, the closest centroid is A. Searching only its two members misses M007 in B; probing A and B recovers the exact top three. This is a small coarse-index demonstration, not HNSW.

Our small demonstration uses three **fixed, authored centroids**. Each movie is assigned to the centroid with highest cosine. The query selects the best one, two, or three cells, and we score eligible members inside them. It illustrates coarse candidate selection; it is not a trained IVF index or an implementation of HNSW.

For `space rescue`, exact cosine finds M004, M008, M007. The nearest cell A contains only M004 and M008. One probe therefore recovers two of the three exact neighbors; a second probe visits B and recovers M007. More probes increase work here. Real speed/quality choices require realistic queries, hardware, filters, and latency measurements.

8. Measure two different kinds of success

Exact-neighbor recall@K asks whether approximate search preserved the exact result under the same vectors, metric, and eligible set. We divide the number of recovered exact neighbors by the size of the exact top-K set. If fewer than K eligible movies exist, that set is smaller; if it is empty, the measure is undefined.

Relevance recall@K, from Chapter 16, asks whether the returned movies answer the search intent. Its denominator is the number of relevant eligible movies. A search can preserve every exact neighbor and still return irrelevant results because the representation or metric is unsuitable.

The authored space example makes exact cosine retrieve all three relevant movies, while BM25 retrieves two. But for `funny adventure`, cosine places Cafe Chaos above Moonlight Map: the nearest movie is funny without being the intended adventure. BM25 places Moonlight Map first. This fixture illustrates possible outcomes, not model performance evidence.

Keep a lexical baseline. On independent judged queries, inspect relevance metrics, missed facts, runtime, memory, and how often ANN loses exact neighbors. Query-specific failure analysis matters more than assuming that newer representation methods always win.

▼ Go deeper: hybrid retrieval and reranking

Hybrid retrieval combines candidates from lexical and vector search. Exact names, rare identifiers, and phrasing can favor lexical matching; paraphrases may favor a learned representation. Raw BM25 and cosine scores have different scales, so simply adding them needs a justified calibration strategy.

Reciprocal rank fusion (RRF) instead sums $1/(c + \text{rank})$ across input lists, using a chosen positive rank constant c . A document absent from a list contributes zero for that list. It combines rank evidence without treating the raw scores as comparable. Candidate cutoffs and c still affect the result; fusion is not a guarantee of improvement. See the [RRF reference](#).

A **cross-encoder reranker** reads a query and candidate together to score their relationship. Joint processing can capture interactions missed by independently encoded vectors, at extra computation per pair. It only reranks retrieved candidates; an omitted source needs better candidate retrieval. See the [retrieve-and-rerank guide](#). Fusion and neural reranking are not implemented in this workbook.

9. Explore the vector search

Start with `space rescue`, cosine, original lengths, and exact search. Inspect the three scores and compare with BM25. Switch to raw dot product: City Sprint jumps to first. Normalize both vectors and compare the three metrics again; their rankings now agree.

Same movies. A different representation.

These three-dimensional vectors were chosen by hand. They illustrate retrieval mechanics; they are not outputs of a trained embedding model. Only three preset queries have supplied vectors.

Space-rescue query $q = [1, 0, 0]$, original vector lengths, $K = 3$, no runtime filter. The default cosine order is M004 (Quiet Orbit), M008 (Parallel World), M007 (Unlisted Voyage).

Search	Results (score)	Neighbor recall@3
Exact cosine	M004 (0.9939) → M008 (0.9810) → M007 (0.9701)	100.00%
Exact raw dot	M002 (3.0000) → M008 (1.6000) → M004 (0.9000)	100.00%
Cosine: 1 cell	M004 (0.9939) → M008 (0.9810)	66.67%
Cosine: 2 cells	M004 (0.9939) → M008 (0.9810) → M007 (0.9701)	100.00%

Exact-neighbor recall uses each row's own metric as its reference. Exact raw dot therefore gets 100% neighbor recall even though City Sprint (M002) is irrelevant to this intent. Raw Euclidean order is M004, M007, M008. Normalize both query and movie vectors: cosine, dot, and Euclidean rankings agree.

Cell A contains M004 and M008; B contains M001, M002, M003, M005, M007; C contains M006. Probing A alone misses M007. Probing A+B recovers the exact top three. BM25 returns M008, M004, M005. Authored vectors make the cosine example work; this is not a measured improvement from a trained encoder.

Reset, then probe only one cell. Neighbor recall drops to 66.67%, and only two result slots are filled. Probe two cells to recover the exact top three. Apply the runtime filter: M007 becomes ineligible, changing both the reference neighbor set and the relevance denominator. A higher filtered score does not mean the old missed movie was found.

The plot shows only two coordinates of three, so it cannot prove distances or semantic clusters. All calculations use every coordinate. The explorer also computes a separate exact scan for its audit; the displayed candidate count describes the selected search stage, not an end-to-end speed benchmark.

10. Run the workbook and practice

Extract all four files together. Python 3.10 or newer and its standard library are enough. No encoder download, network call, API key, or third-party package is required.

```
py vector_search.py
py vector_search.py --metric dot
py vector_search.py --metric euclidean --normalization unit
py vector_search.py --mode probe1
py vector_search.py --query funny --k 1
py -m unittest -v test_vector_search.py
```

Use `python` or `python3` if that is your installed command. The script reads `vectors.json` beside itself and prints JSON without modifying files. `--limit under120`, `--k`, and `--input` control eligibility, result count, and the snapshot. Invalid inputs exit with code 2; approximate modes require cosine. The BM25 comparison uses the Chapter 16 formula and the same summaries.

Exercise 1: calculate a neighbor's score

For $q = [1, 0, 0]$ and Unlisted Voyage's $x = [0.8, 0.2, 0]$, calculate dot product, norm, cosine, and Euclidean distance. Which of these is a probability?

▼ Reveal the reasoning

Dot = 0.8; norm = $\sqrt{0.68} \approx 0.8246$; cosine = $0.8/\sqrt{0.68} \approx 0.9701$; distance = $\sqrt{0.08} \approx 0.2828$. None is a calibrated probability of relevance. Their meanings come from geometry and the representation contract.

Exercise 2: audit an approximate result

The exact top three are M004, M008, M007. A one-cell search returns M004 and M008. Compute neighbor recall@3 and relevance Precision@3 when all three exact neighbors are judged relevant. Then apply the runtime filter that excludes M007.

▼ Reveal the reasoning

Initially both measures are 2/3. With the filter, the exact top three become M004, M008, M003; M003 is another eligible neighbor but is nonrelevant to this intent. One-cell search still returns only M004 and M008: neighbor recall stays 2/3, relevance Precision@3 stays 2/3, and relevance recall rises to 2/2. The denominators answer different questions. Probe more cells to recover the filtered exact reference.

Exercise 3: change magnitude without changing direction

In a copy of the snapshot, multiply every coordinate of M007 by 10. Predict what changes under exact cosine and raw dot product, then run both. What happens after unit normalization?

▼ Reveal the reasoning

Positive scaling preserves direction: cosine and cosine-based cell assignment stay the same apart from rounding. M007's raw dot product for the space query rises from 0.8 to 8, making it first. Unit normalization removes that scale change, so normalized rankings are preserved. Euclidean distances from the raw query generally change. Restore the original snapshot to reproduce handbook numbers.

11. Check your understanding

01 Two encoders output 384 numbers each. Can their vectors automatically be compared?

- A. Yes; equal dimensions guarantee the same space.
- B. Only if their query/document representations are compatible; dimension alone is insufficient.
- C. Only after sorting each vector's coordinates.

Answer: B. Each encoder defines its own coordinate system and training objective. Use a documented compatible pair and the correct preprocessing.

02 $\mathbf{q} = [1, 1]$, $\mathbf{A} = [1, 1]$, $\mathbf{B} = [2, 2]$. Which statement is correct?

- A. Cosine ties A and B; raw dot product gives B twice A's score.
- B. Cosine always prefers the longer vector.
- C. All three metrics necessarily give the same values.

Answer: A. The directions are identical. Dot products are 2 and 4; cosine is 1 for both. Euclidean distances are 0 and the square root of 2.

03 What happens when both nonzero vectors are normalized to unit length?

- A. Their coordinates become probabilities.
- B. The representation becomes factually correct.
- C. Dot product equals cosine, and squared Euclidean distance equals $2 - 2 \times \text{cosine}$.

Answer: C. The geometric identity makes the rankings agree for a fixed query, apart from numerical ties. It does not establish semantic usefulness.

04 An approximate search recovers every exact top-K neighbor. What is established?

- A. Every returned movie is relevant to the person.
- B. The search preserved the reference neighbors under the same metric and eligible set.
- C. The embedding model is perfect.

Answer: B. Neighbor recall compares retrieval algorithms on the same vectors. Relevance evaluation uses judgments about the actual task.

05 Which statement best describes this chapter's vectors?

- A. They were generated by a production neural encoder.
- B. They prove embeddings always beat BM25.
- C. They are hand-authored fixtures that make the arithmetic and search tradeoffs inspectable.

Answer: C. The search and scores are real calculations, but the coordinates are teaching choices. A learned encoder needs its own evidence and evaluation.

12. Bring the source text back into the system

You can now distinguish **encoder quality**, **vector geometry**, **index search quality**, and **task relevance**. A nearest-neighbor result is a set of source IDs and scores; a learning assistant still needs the actual passages, their provenance, and appropriate checks before making claims.

Embeddings can help find related text. They do not prove that a movie meets every constraint, that a source is current, or that a generated answer follows the source faithfully. Keep metadata, evidence, and evaluation beside the vectors.

Chapter 18: Building a RAG system step by step connects ingestion, passage preparation, retrieval, context assembly, and a grounded answer. This chapter's compatible-space contract and keyword baseline will remain useful throughout that pipeline.

▼ Quick-reference glossary

Term	Meaning here
Embedding / encoder	Numerical representation / function producing it
Dimensions / norm	Number of coordinates / vector length
Bi-encoder	Represent query and document independently for comparison
Cosine / dot / Euclidean	Direction comparison / inner product / coordinate distance
Unit normalization	Divide a nonzero vector by its norm
Vector index	Structure for finding neighbors among stored vectors
ANN / exact search	Approximate neighbor lookup / score every eligible vector
Centroid / probe	Cell representative / visiting a selected cell
Neighbor recall	Agreement with the exact top-K reference
Relevance recall	Coverage of judged relevant eligible items
Hybrid / reranker	Combine retrieval methods / rescore retrieved candidates

Sources and further reading

The primary paper and implementation guides are linked beside the relevant concepts. All movie coordinates and centroids in the explorer are authored fixtures; the workbook calculates the scores, index memberships, and evaluations. No learned embedding quality, neural reranking result, or production latency is claimed from this example.